Rapport d’industrialisation — Prédiction du churn

Auteur : Deschant Kounou
Formation : M2 — Industrialisation du développement, ESMT
Date : 9 novembre 2026
Dépôt : https://github.com/deschantkn/ml-churn-project

1. Contexte et objectif

Un opérateur télécom veut repérer les clients qui risquent de résilier leur abonnement (churn), pour agir avant leur départ. Un modèle de Machine Learning peut faire cette prédiction, mais un modèle qui ne vit que dans un notebook ne sert à personne : il faut pouvoir l’appeler, le tester, le déployer et le surveiller.

L’objectif du projet est donc de faire passer le modèle de l’environnement de Data Science à une **application industrielle** : déployable, testable, monitorée et maintenable.

2. Démarche

Le projet a suivi le cycle complet, une brique à la fois. Chaque brique s’appuie sur la précédente.

Étape	Ce qui a été fait	Résultat
Données	Dataset synthétique de 2 000 clients, 6 variables, graine fixe	`data/raw/churn.csv`, reproductible
Exploration	Statistiques, valeurs manquantes, répartition du churn, corrélations	Pas de valeur manquante, 43 % de churn
Modélisation	Comparaison de 3 modèles, sauvegarde du Random Forest	`models/model.pkl`
API	Service FastAPI : santé, informations du modèle, prédiction	Modèle appelable en HTTP/JSON
Interface	Formulaire web servi par l’API	Utilisable sans outil technique
Tests	pytest (API), Selenium (E2E), JMeter (charge)	9 tests automatiques au vert
Versionnement	Git, branches `main` / `develop` / `feature/*`	Historique propre, travail isolé par fonctionnalité
Conteneurisation	Dockerfile et Docker Compose	Même environnement partout
CI	GitHub Actions : tests, E2E, build Docker	Chaque push est vérifié automatiquement
Logs	Logs JSON → Logstash → Elasticsearch → Kibana	Chaque prédiction est traçable

Étape	Ce qui a été fait	Résultat
Monitoring	`/metrics` → Prometheus → Grafana	Trafic, latence et erreurs visibles en direct

3. Résultats

3.1 Modèle

Modèle	Accuracy	Precision	Recall	F1
Régression logistique	0,625	0,582	0,453	0,510
Arbre de décision	**0,632**	**0,583**	**0,512**	**0,545**
Random Forest (retenu)	0,600	0,543	0,436	0,484

Les scores sont modestes, ce qui est attendu : le dataset est synthétique et le churn y est en grande partie aléatoire. Le point important est discuté en section 5 : le modèle imposé par le sujet n'est pas le meilleur des trois.

3.2 Application

- L'API répond sur toutes ses routes, et une prédiction prend environ 15 à 50 ms (mesuré pendant les tests).
- Les entrées invalides sont refusées avec un code 422, sans faire planter le service.
- La stack complète (API, Elasticsearch, Logstash, Kibana, Prometheus, Grafana) démarre avec une seule commande : `docker compose up -d`.
- Chaque prédiction apparaît dans Kibana quelques secondes plus tard, avec ses entrées, son résultat et son temps de réponse.
- Le tableau de bord Grafana se charge tout seul au démarrage, sans configuration manuelle.

3.3 Tests de charge (JMeter)

Plan : `POST /predict`, montée en charge sur 10 secondes, 10 requêtes par utilisateur. API lancée dans son conteneur Docker (un seul processus Uvicorn), sur un Mac à 12 cœurs, le 9 novembre 2026.

Utilisateurs	Requêtes	Moyenne (ms)	Médiane (ms)	90e centile (ms)	95e centile (ms)	Débit (req/s)	Erreurs (%)
100	1 000	1 234	887	2 646	2 826	35,7	0
500	5 000	18 405	19 820	20 919	21 179	24,8	0
1 000	10 000	35 825	38 312	39 357	39 540	26,0	0

Mesures obtenues avec `./tests/jmeter/run_load_tests.sh`. Les rapports HTML détaillés sont générés dans `tests/jmeter/results/`.

Analyse :

- **Aucune erreur**, même à 1 000 utilisateurs : les 16 000 requêtes ont toutes reçu une réponse 200. Le service reste stable sous charge.
- **Le débit plafonne autour de 25 requêtes par seconde**, quel que soit le nombre d'utilisateurs. C'est la capacité maximale d'un seul processus Uvicorn qui exécute un Random Forest de 200 arbres : chaque prédiction occupe le processeur environ 40 ms, et les requêtes sont traitées une par une.
- **Le temps de réponse augmente avec la file d'attente, pas avec le calcul.** Une requête isolée prend environ 15 à 50 ms. Sous charge, elle attend son tour : à 1 000 utilisateurs, la file contient environ 1 000 requêtes, traitées à 26 par seconde, d'où environ 38 secondes d'attente (médiane : 38,3 s).
- **La dégradation commence dès 100 utilisateurs** (médiane proche de 0,9 s). À 500 utilisateurs et plus, le service n'est plus utilisable de façon interactive : une réponse en 20 à 40 secondes est trop lente.

Conclusion des tests de charge : l'API est fiable mais pas dimensionnée pour plus de quelques dizaines de requêtes par seconde. Trois pistes pour augmenter le débit, détaillées en section 6 : lancer plusieurs processus Uvicorn (`--workers`) pour utiliser tous les cœurs ; n'appeler le modèle qu'une fois par requête (`predict_proba` suffit à déduire la classe, alors que l'API appelle aujourd'hui aussi `predict` , ce qui double le calcul) ; et ajouter des conteneurs derrière un répartiteur de charge.

---

4. Difficultés rencontrées

Problème	Cause	Solution
Push refusé par GitHub : *refusing to allow an OAuth App to create or update workflow*	Le jeton utilisé par Git n'avait pas le droit `workflow` , obligatoire pour modifier `.github/workflows/`	Ajout du droit avec `gh auth refresh -s workflow` , puis `gh auth setup-git` pour que Git utilise ce jeton
La CI risquait d'échouer à l'installation	La CI du sujet utilisait Python 3.11, alors que le Dockerfile et `requirements.txt` ont été faits en Python 3.14	Même version (3.14) dans la CI et dans l'image
Les tests Selenium échouaient quand l'API n'était pas lancée	Le test a besoin d'un vrai serveur et d'un navigateur	Test ignoré si l'API ne répond pas ; dans la CI, une étape dédiée démarre l'API avant de lancer Selenium en mode headless
Les logs arrivaient dans Elasticsearch sans être décodés (`_jsonparsefailure`)	Le champ `event` des logs entrait en conflit avec l'objet `event` que Logstash ajoute lui-même	Champ renommé en `operation`
Pas d'interface web pour le scénario Selenium	Le sujet décrit un parcours utilisateur, mais l'API n'avait que Swagger	Ajout d'un formulaire simple, servi par l'API sur `/ui`

---

5. Limites

1. **Le modèle déployé n'est pas le meilleur.** Le Random Forest a le F1 le plus bas (0,484 contre 0,545 pour l'arbre de décision). Dans un vrai projet, le choix du modèle se ferait sur les métriques, et le script d'entraînement sauvegarderait automatiquement le meilleur.
2. **Le recall est faible (0,44).** Le modèle rate plus de la moitié des clients qui partent. Pour un opérateur, rater un départ coûte souvent plus cher qu'une fausse alerte, donc ce point compte.
3. **Les données sont synthétiques.** Les résultats ne disent rien de la performance sur de vrais clients.
4. **Le modèle est figé dans l'image Docker.** Changer de modèle oblige à reconstruire et redéployer toute l'image.
5. **L'API n'a pas d'authentification.** N'importe qui peut l'appeler et lire les métriques.
6. **Elasticsearch tourne sans sécurité.** C'est acceptable en TP, pas en production.
7. **Le déploiement reste manuel.** La CI teste et construit l'image, mais ne la déploie pas.

---

6. Améliorations proposées

Priorité	Amélioration	Gain attendu
Haute	Sélectionner le modèle automatiquement sur le F1 ou le recall, et ajuster le seuil de décision	Meilleures prédictions, choix justifié
Haute	Suivre les entraînements et versionner les modèles (MLflow)	Savoir quel modèle tourne, et revenir en arrière facilement
Haute	Lancer plusieurs processus Uvicorn (`--workers`)	Débit plus élevé sous charge
Moyenne	Déploiement continu : publier l'image dans un registre, puis déployer automatiquement	Moins d'erreurs manuelles
Moyenne	Surveiller la dérive des données (comparer les entrées reçues aux données d'entraînement)	Savoir quand réentraîner
Moyenne	Ajouter des alertes (Grafana ou Alertmanager) sur le taux d'erreur et la latence	Réagir avant les utilisateurs
Basse	Authentification par clé d'API, sécurité activée sur Elasticsearch	Service utilisable en production

---

7. Conclusion

Le modèle de churn est passé d'un script à une application complète : une API documentée, une interface web, des tests à trois niveaux (API, navigateur, charge), une image Docker, une CI qui vérifie chaque modification, et une observabilité complète par les logs (ELK) et les métriques (Prometheus et Grafana).

Le projet montre surtout que l’industrialisation dépasse le modèle lui-même : la plupart du travail porte sur ce qui l’entoure. Il montre aussi que cette chaîne rend les problèmes visibles. Le choix discutable du Random Forest, par exemple, ressort clairement dès qu’on compare les métriques. La prochaine étape logique est de fermer la boucle : choisir et versionner le modèle automatiquement, puis le redéployer quand les données changent.